

INTEGRATED MASTER ROADMAP · RTX 3050 4GB

# CUDA LLM MASTERY

// Project-first. Week-by-week. Every deliverable is concrete.  
Merged from 3 roadmaps – nothing redundant, nothing missing.

■ RTX 3050 4GB · GA106 Ampere · 80 Tensor Cores · 128-bit bus | 8GB System RAM

|                |              |                     |                   |  |
|----------------|--------------|---------------------|-------------------|--|
| 12<br>PROJECTS | ~30<br>WEEKS | 3<br>SOURCES MERGED | ↑ ↑<br>DIFFICULTY |  |
|----------------|--------------|---------------------|-------------------|--|

---

**The rule:** Each project comes first. Weeks tell you exactly what to ship.

"You'll need to figure out" = concepts you'll hit a wall on. Look them up then, not before.

Resources are lookup references – not a reading list to complete before starting.

# 00 Speed of Light Benchmark

Week 1 · Know your ceiling before writing a single kernel

Starter



PROJECT 00

## Benchmark Your RTX 3050 – Peak TFLOPS & Memory Bandwidth

Before writing any kernel, measure your hardware's physical ceiling. You'll reference these numbers in every future project to know if you're hitting the roof.

### WEEK 1 Setup + measure hardware limits

- Install CUDA toolkit, NVCC, Nsight Systems + Compute on WSL2/Ubuntu
- Write a bandwidth test: time cudaMemcpy of 1GB host → device, compute GB/s
- Write a compute test: max-throughput fp32 kernel, compute TFLOPS achieved
- Record your 3050's actual numbers: theoretical vs achieved bandwidth and TFLOPS
- Write a one-page "Hardware Spec Sheet" — you'll reference this all roadmap

#### YOU'LL NEED TO FIGURE OUT

NVCC compilation flags

cudaEvent\_t timing

theoretical peak =  $\text{SMs} \times \text{clock} \times \text{ops/cycle}$

PCIe bandwidth vs VRAM bandwidth

TGP throttling on laptop GPUs

◆ [CUDA Install Guide \(Linux/WSL2\)](#)

◆ [Nsight Systems](#)

◆ [Nsight Compute](#)

◆ [RTX 3050 Mobile Specs \(TechPowerUp\)](#)

# 01 The Memory Architect — SGEMM

Weeks 2–3 · Global mem → shared mem → beat cuBLAS

---

Starter



## PROJECT 01

### Tiled SGEMM – Reach 60–70% of Your 3050's Peak TFLOPS

Write a raw CUDA matrix multiply without cuBLAS. Naive version first — it will be 10× slower than it should be. Profile, understand why, fix it.

#### WEEK 2 Naive kernel + profiling baseline

- Write naive GEMM: one thread = one output element, global memory only
- Test correctness vs numpy matmul (fp32, tolerance 1e-4)
- Profile with ncu — identify it as "memory bound", record L2 cache hit rate
- Compute arithmetic intensity of your naive kernel (FLOPS / bytes moved)

#### WEEK 3 Tiled kernel + benchmark table

- Implement tiled GEMM: load tiles into `__shared__` memory, compute locally
- Fix bank conflicts: pad shared memory arrays, verify with ncu bank conflict counter
- Benchmark table: naive vs tiled vs cuBLAS at sizes 512<sup>2</sup>, 1024<sup>2</sup>, 4096<sup>2</sup>
- README: annotated diagram of the tiling strategy + your TFLOPS numbers

#### WEEK 3+ Read the PTX your kernel generates [↓ LOWER LEVEL](#)

- Dump PTX for both naive and tiled kernels: `nvcc -ptx mykernel.cu -o mykernel.ptx`
- Find `ld.global` vs `ld.shared` — confirm shared memory loads appear in inner loop
- Look for `st.local` / `ld.local` — if present, you have a register spill, fix it
- Check `bar.sync` count — matches your `__syncthreads()` calls
- Add `const __restrict__` to kernel args, redump PTX — spot `ld.global.nc` appearing (L1 texture cache for read-only)
- Use Godbolt with CUDA NVCC + flag `-ptx` to browse PTX interactively

```
// what you want to see in your tiled kernel's inner loop:
ld.shared.f32    %f1, [%r4];          // load from shared – good
ld.shared.f32    %f2, [%r5];          // load from shared – good
fma.rn.f32       %f3, %f1, %f2, %f3; // fused multiply-add – good

// what you do NOT want to see:
ld.global.f32    %f4, [%rd1];         // global load in inner loop – not tiled properly
st.local.f32     [%rd2+0], %f5;       // register spill – too many variables
```

YOU'LL NEED TO FIGURE OUT

thread/block/grid hierarchy

\_\_shared\_\_ memory lifetime

\_\_syncthreads()

memory coalescing

shared memory bank conflicts

occupancy vs register count

roofline model

PTX ld.global vs ld.shared

register spill → st.local

♦ [CUDA Guide – Shared Memory](#)

♦ [Simon Boehm – How to Optimize CUDA Matmul](#)

♦ [NVIDIA Blog – Using Shared Memory](#)

♦ [CUDA by Example – Ch3-5](#)

♦ [PTX ISA Reference \(use as dictionary\)](#)

♦ [Godbolt – Compiler Explorer \(CUDA NVCC + --ptx\)](#)

## 02 The Warp Tactician — Parallel Reduction

Week 4 · Master warp-level primitives

Starter → Mid



### PROJECT 02

#### Sum 100M Elements – No Shared Memory Allowed in Final Version

Sum a 100-million element float array. Start with shared memory reduction, then eliminate it entirely using warp shuffle intrinsics. This unlocks softmax, layernorm, and attention.

#### WEEK 4 Tree reduction → warp shuffle version

- Naive reduction: divergent if/else, profile warp divergence in ncu
- Tree reduction: eliminate divergence, benchmark speedup
- Warp shuffle version: replace \_\_shared\_\_ with \_\_shfl\_down\_sync entirely
- Add atomic final reduction across blocks
- Benchmark all 3 versions — chart showing GB/s for each
- Bonus: implement parallel max (needed for softmax)

YOU'LL NEED TO FIGURE OUT

warp divergence cost

\_\_shfl\_down\_sync mask

warp = 32 threads always execute together

atomicAdd limitations

loop unrolling #pragma unroll

block-level vs warp-level reduction

♦ [Mark Harris – Optimizing Parallel Reduction \(NVIDIA\)](#)

♦ [CUDA Guide – Warp Shuffle Functions](#)

♦ [NVIDIA Blog – Warp-Level Primitives](#)



## PROJECT 03

**Gaussian Blur + Sobel Filter – Visual Output You Can See**

Implement 2D convolution on a real image. The boundary pixel "halo" problem forces you to handle warp divergence in a concrete, debuggable way. Teaches constant memory — a cache type LLMs also use for weight broadcasts.

## WEEK 5 Convolution kernel + constant memory

- Naive 2D convolution kernel on a 4K image (Gaussian 5×5 kernel)
- Move filter weights to `__constant__` memory — measure speedup
- Implement Sobel edge detection as second kernel
- Handle halo/border pixels without warp divergence (predication instead of if/else)
- Save output images — visual proof your kernels work
- Profile: compare warp efficiency before/after divergence fix in ncu

## YOU'LL NEED TO FIGURE OUT

`__constant__` memory 64KB limit

broadcast vs cache difference

predication over branching

2D block/grid indexing

texture memory (optional)

stb\_image for loading images in C

[♦ CUDA Guide – Constant Memory](#)[♦ NVIDIA – Grid-Stride Loops](#)[♦ stb\\_image.h \(single-header image loader\)](#)

# 04 The Tensor Core Specialist — WMMA

Weeks 6–7 · Use your 3050's actual AI hardware

---

Mid



## PROJECT 04

### FP16 GEMM with Tensor Cores – 4× Faster Than Your FP32 Kernel

Your RTX 3050 has 80 Tensor Cores that your P01 kernel never touched. Use `nvcuda::wmma` to explicitly trigger them. This is what PyTorch uses under the hood for every LLM matmul.

#### WEEK 6 FP16 data layout + WMMA fragments

- Convert FP32 matrices to FP16 (`__half` type), verify precision loss acceptable
- Understand `wmma::fragment` layout — how 16×16 tiles map to warp registers
- Implement basic `wmma::mma_sync` call — smallest working Tensor Core kernel
- Correctness test vs FP32 cuBLAS (allow fp16 tolerance)

#### WEEK 7 Tiled WMMA GEMM + benchmark vs P01

- Full tiled WMMA GEMM: 16×16 Tensor Core tiles + shared memory staging
- Benchmark: FP32 tiled (P01) vs FP16 WMMA — measure TFLOPS both
- Profile Tensor Core utilization in `ncu` — it has a dedicated counter
- README: explain why Tensor Cores need fp16 and what "mixed precision" means

#### WEEK 7+ Read SASS – confirm Tensor Cores actually fired [↓ LOWER LEVEL](#)

- Dump SASS (actual binary ISA) with: `cuobjdump --sass mykernel.cubin`
- Find `HMMMA` instructions — these are your Tensor Core calls. Count them in the inner loop
- Check for `LDG` (load global), `LDS` (load shared), `STL` (store local = spill)
- Compare SASS of your WMMA kernel vs your P01 FP32 kernel — spot the structural difference
- Rewrite one kernel using the CUDA Driver API directly ( `cuLaunchKernel` instead of `<<<>>>` )
- Enable driver tracing: run with `CUDA_LAUNCH_BLOCKING=1` + `nsys`, observe every driver call in timeline

```
// dump SASS from compiled binary:
nvcc -cubin mykernel.cu -o mykernel.cubin
cuobjdump --sass mykernel.cubin | grep -A5 "Function : _Z"

// what Tensor Core firing looks like in SASS:
HMMMA.16816.F32 R4, R8, R12, R4 // Tensor Core: 16×8×16 mixed-prec matmul
LDS.128 R8, [R2+0x80] // 128-bit shared load – vectorized, good

// bad signals:
STL [R1+0x10], R5 // register spill to local mem
LDG.E.SYS R4, [R2] // uncached global load
```



#### YOU'LL NEED TO FIGURE OUT

\_\_half vs float precision trade-offs

wmma::fragment memory layout

16×16×16 tile requirement

load\_matrix\_sync / store\_matrix\_sync

mixed precision: fp16 compute, fp32 accumulate

Tensor Core occupancy requirements

HMMA instruction in SASS

cuLaunchKernel driver API

◆ [CUDA Guide – WMMA](#)

◆ [NVIDIA Blog – Programming Tensor Cores](#)

◆ [CUTLASS repo](#)

◆ [CUDA Driver API – cuLaunchKernel](#)

◆ [cuobjdump / SASS reference](#)

## 05 The Kernel Fusionist — Softmax + LayerNorm

Mid

Weeks 8–9 · Ops that run in every LLM forward pass



### PROJECT 05

#### Fused Softmax + LayerNorm – Beat PyTorch Baseline

Both ops appear in every transformer block. Implement them separately, then fuse into one kernel launch. Python bindings so you can call them from PyTorch. Backward pass required.

#### WEEK 8 Standalone kernels + Python bindings

- Numerically stable softmax kernel using online max + sum (Milakov algorithm)
- LayerNorm kernel: mean, variance in one pass using warp reduction from P02
- Both exposed as torch custom ops via torch.utils.cpp\_extension
- Unit tests: outputs match torch.nn.functional to 1e-5

#### WEEK 9 Fusion + backward pass + benchmark

- Fused single-kernel version: softmax → layernorm, one HBM round-trip
- Backward pass for LayerNorm (derive the gradient by hand first)
- Benchmark: unfused vs fused, measure GB/s reduction
- Profile: show HBM bytes read/written before and after fusion in ncu

#### YOU'LL NEED TO FIGURE OUT

online softmax (running max trick)

kernel fusion = fewer HBM round-trips

register pressure with many variables

torch.utils.cpp\_extension setup

backward pass chain rule for layernorm

numerical stability in fp16

◆ [Online Softmax paper \(Milakov 2018\)](#)

◆ [PyTorch Custom CUDA Extensions](#)

◆ [Triton LayerNorm tutorial \(reference\)](#)

◆ [Mark Harris – Parallel Reduction \(reuse from P02\)](#)

# 06 The Most Important Kernel — FlashAttention

Weeks 10–11 · Raw CUDA, no shortcuts

---

Hard



## PROJECT 06

### FlashAttention Kernel – Tile Q/K/V to Fit in Your 3050's SRAM

Standard attention materializes an  $N \times N$  matrix in VRAM — impossible for long sequences on 4GB. Tile Q/K/V blocks into shared memory, compute online softmax, never write the intermediate matrix. Profile HBM bytes — that's the whole point.

#### WEEK 10 Forward pass: tiled attention + online softmax

- Implement naive attention in CUDA first — profile its HBM usage as baseline
- Implement tiled FA-style forward: Q/K/V loaded in blocks, online softmax accumulation
- Causal masking inside the tiled kernel
- Correctness test vs naive PyTorch attention at seq\_len 128, 512, 1024

#### WEEK 11 Profile HBM savings + written explanation

- Nsight profile: HBM bytes read — naive vs tiled (should be dramatically less)
- Plot: sequence length vs peak VRAM for naive vs FlashAttention
- Written explanation in your own words: why does tiling save memory?
- Bonus: backward pass — recompute softmax stats from saved {m, l} instead of storing activations

#### WEEK 11+ Warp stall breakdown + first chip design reading ↓ LOWER LEVEL

- Run ncu with full warp stall sections — this shows exactly why warps are idle
- Identify your dominant stall type: is it memory latency, sync barriers, or instruction pipeline?
- For each stall type, understand what the hardware scheduler is waiting on
- Read the Chips and Cheese Ampere article — understand the GA106 die your code runs on
- Write a paragraph connecting your stall reason to the actual hardware unit causing it

```
ncu --section WarpStateStatistics \  
    --section MemoryWorkloadAnalysis \  
    --section ComputeWorkloadAnalysis \  
    ./myflashattention  
  
// WarpStateStatistics shows you why warps aren't issuing instructions.  
// The most common stall reasons and what they mean:  
  
Stall_Long_Scoreboard // waiting for global memory load – you're memory bound  
Stall_MIO_Throttle    // memory instruction queue full – too many mem ops in flight  
Stall_Barrier         // hitting __syncthreads() – reduce sync points  
Stall_No_Instruction  // pipeline empty, all warps busy computing – you're compute bound (rare, good)  
Stall_Short_Scoreboard // waiting for shared mem / math unit – short latency, ok
```

#### YOU'LL NEED TO FIGURE OUT

HBM vs SRAM bandwidth gap (~10×)

online softmax with running {m, l}

tile size constraints (SRAM limit)

register spilling to local memory

causal mask in tiled loop

recompute vs store trade-off

Stall\_Long\_Scoreboard = memory latency

Stall\_Barrier = too many syncthreads

◆ FlashAttention-2 paper (Tri Dao)

◆ FlashAttention repo (CUDA kernels)

◆ ELI5 FlashAttention – Aleksa Gordić

◆ Online Softmax paper (Milakov 2018)

◆ Chips and Cheese – Ampere In-Depth

◆ ncu Warp States Reference

## 07

## The Quantization Engineer — INT4 Bitpacking

Mid

Week 12 · Bitwise ops, vectorized loads, Tensor Core dequant



### PROJECT 07

#### FP16 → INT4 Quantization Kernel – Pack & Unpack at Full Speed

Pack four INT4 values into a single 16-bit integer. Write the dequantization kernel that runs during inference. Use float4 vectorized 128-bit loads. This is exactly how GPTQ and AWQ work under the hood.

#### WEEK 12 Quantize kernel + dequant on-the-fly + benchmark

- Quantization kernel: fp16 → INT4 with per-channel scale + zero-point
- Bitpacking: store 4× INT4 values in one uint16\_t using shifts and masks
- Dequantization kernel: unpack INT4 → fp16 on-the-fly during matmul
- Use float4 vectorized loads (128-bit) — verify memory throughput improves
- Benchmark: INT4 weight matmul vs FP16 — memory bandwidth and latency
- Quality check: measure RMSE of reconstructed weights vs originals

#### YOU'LL NEED TO FIGURE OUT

bitwise <<, >>, & in CUDA

per-channel vs per-tensor quantization

float4 / uint4 vectorized types

absmax calibration

why INT4 saves memory but not always compute

\_half2 operations

◆ LLM.int8() paper (Dettmers)

◆ AWQ paper

◆ GPTQ paper

◆ CUDA Guide – Vector Types (float4)

# 08 Train a Real Language Model

Weeks 13–15 · Every tensor shape must make sense to you

Mid



## PROJECT 08

### GPT-2 124M from Scratch – fp16 + Grad Checkpointing on Your 3050

No tutorial. Read nanoGPT once, close it, write your own. Train on a real dataset. Use fp16 AMP and gradient checkpointing — required on 4GB VRAM. Sequence length 256, batch size 2.

#### WEEK 13 Architecture: attention, FFN, embeddings

- Implement multi-head attention, causal mask, QKV projections from scratch
- FFN with GELU activation, pre-LayerNorm (GPT-2 style)
- Token + positional embeddings, weight tying (embedding = lm\_head)
- Forward pass test: random inputs produce correct output shape, no NaNs

#### WEEK 14 Training loop + memory optimizations

- AdamW optimizer, cosine LR schedule, gradient clipping (norm 1.0)
- fp16 AMP with torch.cuda.amp.autocast — verify no overflow
- Gradient checkpointing: torch.utils.checkpoint on transformer blocks
- Verify it fits in 4GB VRAM at seqlen=256, batch=2
- Checkpoint save/load: resume from any step

#### WEEK 15 Train on real data + generate text

- Download + tokenize a dataset (TinyShakespeare or OpenWebText 1%)
- Train until loss visibly decreases — doesn't need to converge fully
- Generate text samples — model should produce coherent-ish sequences
- Annotated tensor shape diagram for one full forward pass
- Memory usage log: peak VRAM at each batch size

#### YOU'LL NEED TO FIGURE OUT

scaled dot-product attention math

causal masking implementation

gradient checkpointing trade-off

fp16 AMP loss scaling

BPE tokenization

AdamW vs Adam difference

memory-mapped datasets on 8GB RAM

♦ nanoGPT (Karpathy)

♦ Attention Is All You Need

♦ Let's build GPT – Karpathy YT

♦ LLaMA 2 paper (architecture reference)

♦ PyTorch Gradient Checkpointing



## PROJECT 09

**Inference Runtime – KV Cache + INT4 Weights + Measure tok/s**

Take your GPT-2 and build an inference runtime. Pre-allocate a KV cache, use your INT4 quantized weights from P07, batch decode. Your metric: tokens per second at various batch sizes.

**WEEK 16** KV cache + autoregressive decode loop

- Pre-allocate KV cache tensors: [batch, n\_heads, max\_seq\_len, head\_dim]
- Decode loop: each step appends to KV cache, attends over full history
- Verify: generated tokens match non-cached generation exactly
- Measure: prefill latency vs decode latency per token

**WEEK 17** INT4 weights + batched decode + benchmark table

- Load INT4 quantized weights (from P07) into inference model
- Batched decode: batch\_size 1, 4, 8 — handle padding correctly
- Benchmark table: tok/s at each batch size, with and without INT4 weights
- Profile: identify the bottleneck — is it memory bandwidth or compute?
- Bonus: implement greedy, top-k, and nucleus sampling

## YOU'LL NEED TO FIGURE OUT

why decode is memory-bandwidth bound

$$\text{KV cache memory growth} = \text{batch} \times \text{heads} \times \text{seqlen} \times \text{head\_dim} \times 2$$

arithmetic intensity of decode step

MQA / GQA (reduces KV cache size)

padding vs packing for batching

continuous batching concept

♦ [vLLM – PagedAttention paper](#)♦ [Anyscale – Continuous Batching](#)♦ [LLM.int8\(\) – Dettmers](#)♦ [GQA paper \(Grouped-Query Attention\)](#)



## PROJECT 10

**Double-Buffered Inference Pipeline – 100% GPU Utilization**

By default, your GPU waits idle while the CPU prepares the next batch. Fix this: use CUDA streams and double buffering so compute on batch N overlaps with data transfer of batch N+1.

## WEEK 18 Streams + pinned memory + double buffer

- Baseline: single-stream pipeline — profile GPU idle time in nsys timeline
- Switch host memory to `cudaHostAlloc` (pinned) — measure PCIe bandwidth improvement
- Two CUDA streams: `stream_a` computes while `stream_b` loads next batch
- Double buffer: two sets of device buffers, ping-pong between streams
- nsys timeline screenshot: GPU compute and memcpy overlapping
- Throughput comparison: samples/sec single-stream vs double-buffered

## YOU'LL NEED TO FIGURE OUT

`cudaStream_t` creation and synchronization`cudaMemcpyAsync` requires pinned memory`cudaHostAlloc` vs `malloc`stream dependency with `cudaStreamWaitEvent`

PCIe bandwidth on laptop (shared with display)

`cudaGraph` for repeated pipelines[♦ NVIDIA – Overlap Data Transfers in CUDA](#)[♦ CUDA Best Practices – Async Transfers](#)[♦ NVIDIA Blog – CUDA Graphs](#)[♦ Nsight Systems \(timeline view\)](#)



## PROJECT 11 — CAPSTONE

**50M LLM with Your Custom Kernels – Train, Quantize, Deploy, Report**

Everything you built feeds into this. Train a 50M model with your FlashAttention kernel and fp16 training. Quantize with your INT4 kernel. Serve with your inference engine and double-buffered pipeline. Write a profiling report.

**WEEKS 19–20** Train 50M model with custom kernels

- Architecture: 50M param transformer (8 layers, 512 dim, 8 heads)
- Swap stock attention for your FlashAttention kernel from P06
- fp16 AMP + gradient checkpointing — must fit in 4GB
- Train on OpenWebText 1% subset — log loss every 100 steps
- Verify: loss curve goes down, model generates plausible text

**WEEKS 21–22** Quantize + serve with full inference stack

- Apply your INT4 quantization kernel from P07 to trained weights
- Measure perplexity: fp16 vs INT4 (should be within 1–2 points)
- Plug into your inference engine (P09): KV cache + batched decode
- Wrap in double-buffered pipeline (P10) for throughput
- Benchmark: tok/s fp16 vs tok/s INT4 at batch sizes 1, 4, 8

**WEEKS 23–24** Profiling report + GitHub publish

- Full Nsight Systems trace: where does time go across the whole system?
- Profiling report: baseline vs each optimization — TFLOPS, GB/s, tok/s
- Published GitHub repo: README, training curves, benchmark tables, architecture diagram
- One-pager: "What I learned about GPU systems from building this"

## YOU'LL NEED TO FIGURE OUT

Triton for any new custom ops

torch.compile interaction with custom kernels

DDP if you ever get a second GPU

perplexity vs downstream eval

OpenWebText streaming to fit in 8GB RAM

♦ [llm.c – Karpathy \(read every file\)](#)♦ [ZeRO paper \(Rajbhandari\)](#)♦ [Triton tutorials](#)♦ [Megatron-LM paper](#)♦ [CUTLASS repo](#)♦ [OpenWebText dataset \(HuggingFace\)](#)



---

# Going Lower — The Full Stack

// Three rabbit holes. Taste each one. Decide how far you want to go.

// Purple weeks inside each project are your entry points. This section is the deeper reading.

## LEVEL 1 · PTX & SASS

### Reading GPU Assembly

PTX is NVIDIA's virtual ISA — what NVCC compiles your C++ to. SASS is the actual binary that runs on your GA106. You'll read both in P01 and P04. The gap between what you wrote and what the hardware executes is where all performance hides.

[♦ PTX ISA Reference](#)[♦ Godbolt – CUDA PTX Explorer](#)[♦ cuobjdump / SASS Utilities](#)

## LEVEL 2 · DRIVER API & RUNTIME

### Below the CUDA Runtime

The CUDA Runtime (`cudaMalloc`, `<<<>>>`) is a convenience wrapper. Below it is the Driver API: explicit context management, module loading, PTX-to-binary JIT compilation at runtime. PyTorch's CUDA backend calls the Driver API directly. Writing one kernel at this level shows you exactly what the runtime hides.

[♦ CUDA Driver API Reference](#)[♦ ncu Profiling Guide – all sections](#)[♦ nsys – full timeline tracing](#)

## LEVEL 3 · MICROARCHITECTURE & HDL

### The Chip Itself

Understanding the GA106 die — SM pipeline, warp scheduler, memory crossbar, Tensor Core dataflow — changes how you think about every kernel. TinyGPU is a minimal GPU in Verilog you can actually simulate. You don't need to write HDL to benefit from reading it.

[♦ Chips & Cheese – Ampere In-Depth](#)[♦ TinyGPU – minimal GPU in Verilog](#)[♦ NVIDIA Ampere Architecture Whitepaper](#)[♦ Patterson & Hennessy – Ch4–5 \(pipelines\)](#)

---

## Week-by-Week at a Glance

### WEEK 1

P00: Hardware benchmark

### WEEK 2

P01: Naive GEMM + profile

### WEEK 3

P01: Tiled GEMM + benchmark

### WEEK 4

P02: Parallel reduction

### WEEK 5

P03: Image convolution



### WEEK 6

P04: FP16 + WMMA fragments

### WEEK 7

P04: Tiled WMMA benchmark

### WEEK 8

P05: Softmax + LayerNorm

### WEEK 9

P05: Fusion + backward

### WEEK 10

P06: FlashAttn forward

### WEEK 11

P06: Profile HBM savings

### WEEK 12

P07: INT4 bitpacking

### WEEK 13

P08: GPT-2 architecture

WEEK 14

P08: Training loop + AMP

WEEK 15

P08: Train on real data

WEEK 16

P09: KV cache + decode

WEEK 17

P09: INT4 + batch benchmark

WEEK 18

P10: Streams + double buffer

WK 19–20

P11: Train 50M model

WK 21–22

P11: Quantize + serve

WK 23–24

P11: Report + publish